

Three questions

Market realism, accessibility, and keeping people from getting discouraged.

Penn Porterfield · 2026 · companion to the thunk proposal

My mentor asked me to address three things head-on. Here they are, honestly, because the honest version is the stronger one.

1 · MARKET REALISM

Is there a job at the end of this? The honest answer has two halves. The direction is real: entry-level web and software work is contracting, with employment for developers in their early twenties down about 20% since 2022², while systems-adjacent engineering holds up and pays more. Web developers earn a median of \$90,930 against \$133,080 for software developers¹, software-developer roles are still projected to grow ~18% through 2033¹, and Rust just became a core part of the Linux kernel.

But I am not going to oversell it. Rust and kernel roles specifically are a smaller slice of the market than web, and no beginner lands a kernel job straight out of a course. So the realistic promise is not “you will be a kernel developer.” It is this: you come out differentiated, with fundamentals most bootcamp graduates do not have, and with tangible artifacts — a device driver you wrote, real open-source contributions — that open a range of roles: embedded, systems, QA, developer tooling, technical support, and general software, where understanding what is under the hood is rare and valued. Employers already treat open-source work as real currency; 83% weigh it when hiring³. The kernel is the ceiling, not the floor. And this does not compete with the web programs. It is a different lane for the people who want one.

2 · ACCESSIBILITY

Who can actually use this, and where? It is built for the least-equipped setting on purpose.

- **No prior programming.** It starts from true zero, self-paced and mastery-based, so nobody is rushed and nobody is locked out.
- **No internet, no install, no admin, no hardware.** It is one static binary, and the whole low-level stack is simulated in software, so it runs on the single locked-down laptop a facility already allows.

- **It clears review.** No network, no cryptography, no hacking language (a check in the build enforces that), so it fits the walled-garden laptops facilities are already deploying.
- **Free and open source.** No cost to a learner or a facility.
- **Two builds.** An air-gapped one for inside and an open one for reentry, so it reaches people on both sides of release.

HONEST GAPS

The open-source part needs the internet, so it lives on the outside build only. And I still owe real work on screen-reader accessibility and, later, languages other than English. I would rather name those than pretend they are solved.

3 · KEEPING PEOPLE FROM GETTING DISCOURAGED

This is the real risk. Systems is hard, and beginners quit when it feels abstract, punishing, or lonely. The design fights all three on purpose.

- **A visible payoff, early and often.** You are not staring at theory for months. You get something on the screen quickly, and the whole course aims at a reward you can see: a game booting on a panel you drove yourself.
- **It cannot get stuck.** Self-paced, help always on screen, no dead ends, no failing grade. A wrong answer just says “not quite, try again.” Not everyone learns at the same pace, and that is the rule here, not a slogan.
- **You are not alone.** The small community means you do your first hard things alongside a few other people, and someone who has already walked the path. Isolation is what kills reentry programs; peers are the fix.
- **The meritocracy reframe.** When your first patch gets picked apart, the community is judging the code, not you. Separating the work from the person takes the sting out of feedback, which is the exact muscle that keeps people going.
- **Momentum you can measure.** Checks passed, lessons done, a driver finished, a patch merged. Small, real wins stacked up, so progress is visible on the days motivation dips.

HONEST NOTE

The kernel community can be blunt. I do not hide that; I prepare people for it and start them on tiny, well-scoped wins instead of the deep end. Bluntness about the code is survivable when it is clearly not about you.

Short version: be honest about the market, remove every barrier to getting in, and make the wins visible and shared so people stay.

SOURCES

1. U.S. Bureau of Labor Statistics, Occupational Outlook / OEWS, 2024: software developers median \$133,080 and ~18% projected growth 2023–33; web developers median \$90,930. [bls.gov/ooh](https://www.bls.gov/ooh)
2. Stanford Digital Economy Lab, “Canaries in the Coal Mine?” (ADP payroll data), 2025: employment for software developers ages 22–25 down ~20% from its 2022 peak.
3. 2024 Open Source Software Funding Survey (GitHub, the Linux Foundation, Harvard): 83% of organizations weigh open-source contributions when hiring.